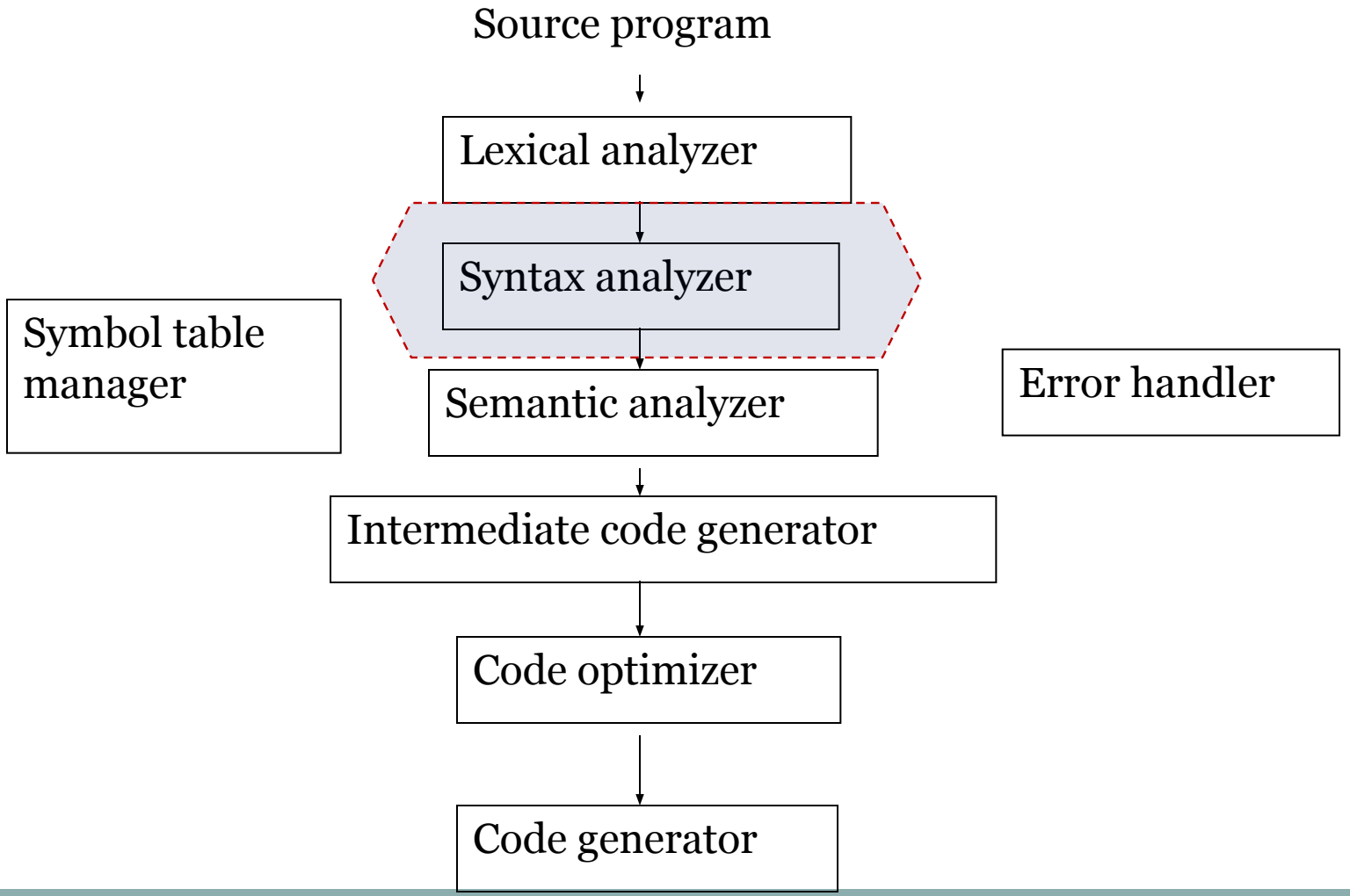
Syntax Analysis

Part I

Phases of a Compiler



Syntax Analyzer

- *Syntax Analyzer* creates the syntactic structure of the given source program.
- This syntactic structure is mostly a *parse tree*.
- Syntax Analyzer is also known as *parser*.
- The syntax of a programming is described by a *context-free grammar (CFG) or* BNF (Backus-Naur Form) notation.

Syntax Analyzer

- The syntax analyzer (parser) checks whether a given source program satisfies the rules implied by a **Context-Free Grammar (CFG)** or not.
  - If it satisfies, the parser creates the parse tree of that program.
  - Otherwise the parser gives the error messages.

A **Context-free Grammar** (**CFG**) is utilized to describe the Syntactic Structure of a Language

A CFG Is Characterized By:

1. A Set of **Tokens** or **Terminal** Symbols (T)
2. A Set of **Non-terminals** or **Variables** (V)
3. A Set of **Production Rules** NT → {T, NT}* (P)
4. A Non-terminal Designated As the **Start Symbol** (S)

Context-Free Grammars

A set of ***terminals***: basic symbols from which sentences are formed

A set of ***nonterminals***: syntactic variables denoting set of strings

A set of ***productions***: rules specifying how the terminals and nonterminals can be combined to form sentences

The ***start symbol***: a distinguished nonterminal denoting the language

CFG: An Example

Terminals: id, +, -, *, /, (,)

Nonterminals: *expr*, *op*

Productions:

expr → *expr op expr*
expr → (*expr*)
expr → - *expr*
expr → id
op → + | - | * | /

The start symbol: *expr*

CFG

- A string of tokens is a sequence of zero or more tokens.
- The string containing with zero tokens, written as $\in$, is called ***empty*** string.
- A grammar derives ***strings*** by beginning with the start symbol and repeatedly replacing the non terminal by the right side of a production for that non terminal.
- The token strings that can be derived from the start symbol form the ***language*** defined by the grammar.

Context-Free Grammars

- In a context-free grammar, we have:
  - A finite set of terminals
  - A finite set of non-terminals
  - A finite set of production rules of the form

    $A \rightarrow \alpha$

    where A is a non-terminal and α is a string of terminals and non-terminals (including the empty string)
  - A start symbol (one of the non-terminal symbol)
- Example:

  $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$

  $E \rightarrow (E)$

  $E \rightarrow id$

Derivations

$E \Rightarrow E+E$

- E+E derives from E
  - we can replace E by E+E

$E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+id$

- A sequence of replacements of non-terminal symbols is called a **derivation** of id+id from E.

Derivations

- In general a derivation step is

  $\alpha A \beta \Rightarrow \alpha \gamma \beta$

  if there is a production rule $A \rightarrow \gamma$ in our grammar where α and β are arbitrary strings of terminal and non-terminal symbols

$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \ldots \Rightarrow \alpha_n$ (α_n derives from α_1 or α_1 derives α_n)

$\overset{*}{\Rightarrow}$: derives in zero or more steps

$\overset{+}{\Rightarrow}$: derives in one or more steps

Derivation Example

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

OR

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(E+id) \Rightarrow -(id+id)$

- At each derivation step, we can choose any of the non-terminal in the sentential form of G for the replacement.
- If we always choose the left-most non-terminal in each derivation step, this derivation is called as **left-most derivation**.
- If we always choose the right-most non-terminal in each derivation step, this derivation is called as **right-most derivation**.

Left-Most and Right-Most Derivations

Left-Most Derivation

$$E \overset{lm}{\Rightarrow} -E \overset{lm}{\Rightarrow} -(E) \overset{lm}{\Rightarrow} -(E+E) \overset{lm}{\Rightarrow} -(id+E) \overset{lm}{\Rightarrow} -(id+id)$$

Right-Most Derivation

$$E \overset{rm}{\Rightarrow} -E \overset{rm}{\Rightarrow} -(E) \overset{rm}{\Rightarrow} -(E+E) \overset{rm}{\Rightarrow} -(E+id) \overset{rm}{\Rightarrow} -(id+id)$$

- We will see
  - the top-down parsers try to find the left-most derivation of the given source program.
  - the bottom-up parsers try to find the right-most derivation of the given source program in the reverse order.

CFG - Terminology

- L(G) is *the language of G* (the language generated by G) which is a set of sentences.
- *A sentence of L(G)* is a string of terminal symbols of G.
- If S is the start symbol of G then

  ω is a sentence of L(G) iff $S \overset{*}{\Rightarrow} \omega$ where ω is a string of terminals of G.

- If G is a context-free grammar, L(G) is a *context-free language*.
- Two grammars are *equivalent* if they produce the same language.

- $S \Rightarrow \alpha$
  - If α contains non-terminals, it is called a *sentential* form of G.
  - If α contains only terminals, it is called a *sentence* of G.

CFG

list → *list* + *digit*

list → *list* - *digit*

list → *digit*

digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

(So we could have written

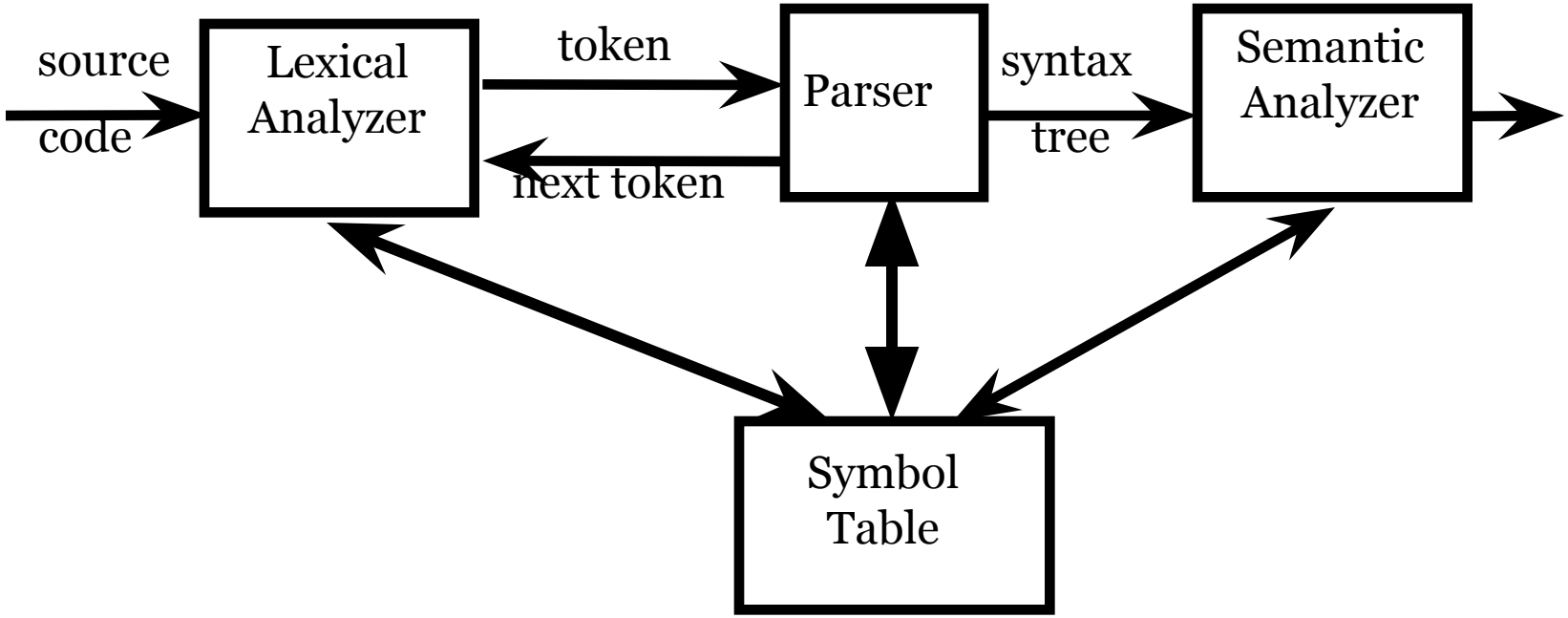
list → *list* + *digit* | *list* - *digit* | *digit*)

Derivation

Using the CFG defined on the earlier slide, we can derive the string: 9 - 5 + 2 as follows:

list → *list* + *digit*	P1 : *list* → *list* + *digit*
→ *list* - *digit* + *digit*	P2 : *list* → *list* - *digit*
→ *digit* - *digit* + *digit*	P3 : *list* → *digit*
→ 9 - *digit* + *digit*	P4 : *digit* → 9
→ 9 - 5 + *digit*	P4 : *digit* → 5
→ 9 - 5 + 2	P4 : *digit* → 2

Parser


source
code
Lexical
Analyzer
token
next token
Parser
syntax
tree
Semantic
Analyzer
Symbol
Table

Parsers (cont.)

Parsers can be categorized into two groups:

Top-Down Parser
- the parse tree is created top to bottom, starting from the root.

Bottom-Up Parser
- the parse tree is created bottom to top; starting from the leaves

- Both top-down and bottom-up parsers scan the input from left to right (one symbol at a time).
- Efficient top-down and bottom-up parsers can be implemented only for sub-classes of context-free grammars.
  - **LL** for top-down parsing (**L**eft to right scan, **L**eft most derivation)
  - **LR** for bottom-up parsing (**L**eft to right scan, **R**ight most derivation)

Parse Trees

- A parse tree pictorially shows how the start symbol of a grammar derives a string in the language.
- More Formally, a Parse Tree for a CFG Has the Following Properties:
  - Root Is Labeled With the **Start Symbol**
  - Leaf Node Is a Token or $\in$
  - Interior Node Is a **Non-Terminal**
  - If $A \rightarrow x1x2\ldots xn$, Then A Is an Interior; $x1x2\ldots xn$ Are Children of A and May be **Non-Terminals** or **Tokens**

Parse Tree

list → *list* + *digit*

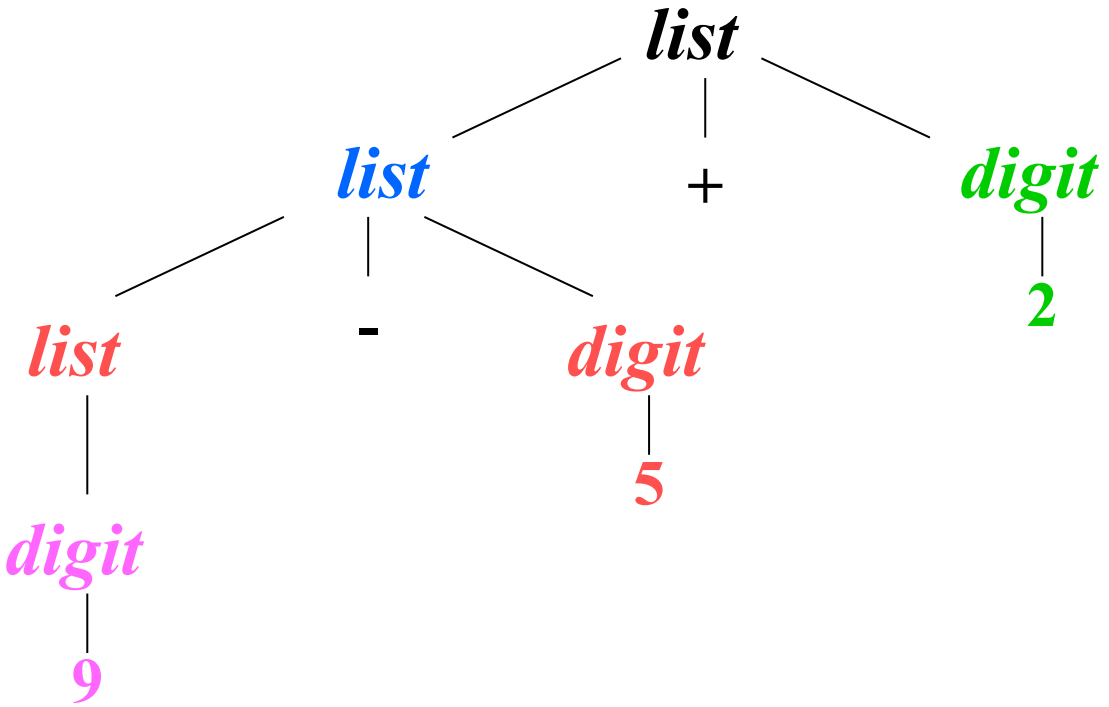
→ *list* - *digit* + *digit*

→ *digit* - *digit* + *digit*

→ 9 - *digit* + *digit*

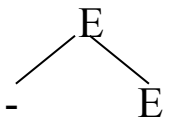
→ 9 - 5 + *digit*

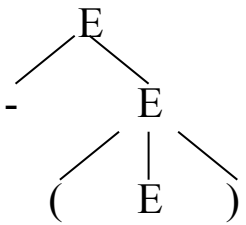
→ 9 - 5 + 2

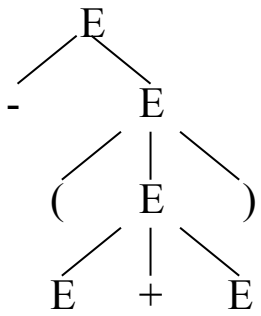


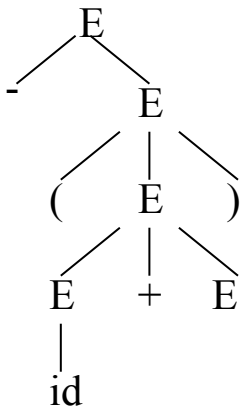
Parse Tree

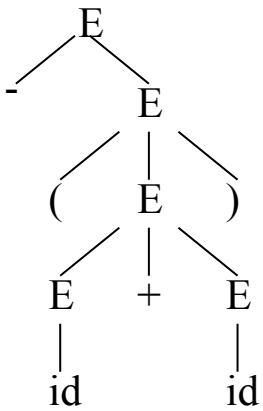
- Inner nodes of a parse tree are non-terminal symbols.
- The leaves of a parse tree are terminal symbols.
- graphical representation of a derivation.

E ⇒ -E 

⇒ -(E) 

⇒ -(E+E) 

⇒ -(id+E) 

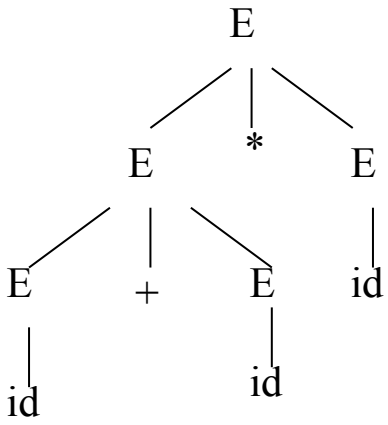
⇒ -(id+id) 

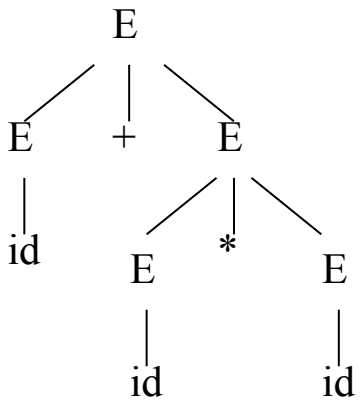
Ambiguity

A grammar produces more than one parse tree for a sentence is called as an ***ambiguous*** grammar.

E → E + E | E – E | E * E | E / E | - E
E → (E)
E → id

E ⇒ E*E
⇒ E+E*E
⇒ id+E*E
⇒ id+id*E
⇒ id+id*id



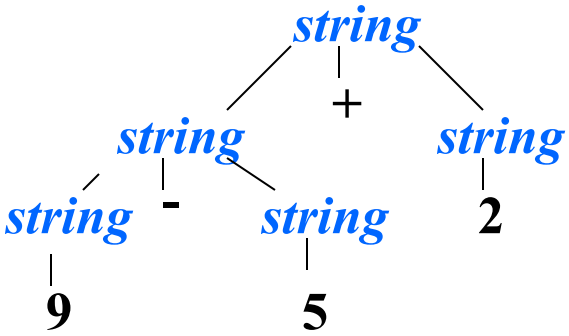


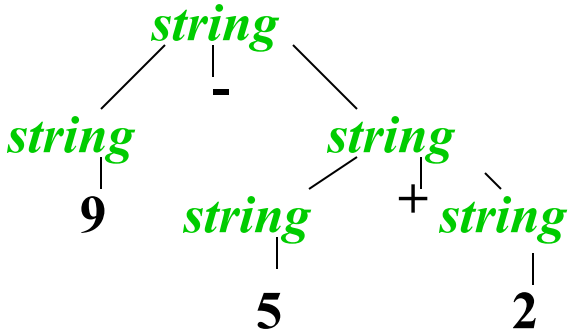
E ⇒ E+E
⇒ id+E
⇒ id+E*E
⇒ id+id*E
⇒ id+id*id

Ambiguity

Grammar:

string → *string* + *string* | *string* – *string* | 0 | 1 | …| 9





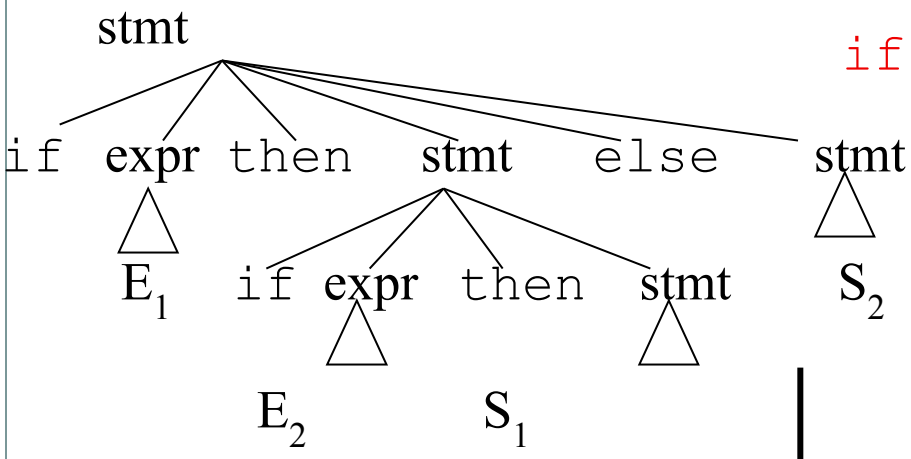
Ambiguity (cont.)

- For most of the parsers, the grammar must be unambiguous.
  - Unambiguous grammar posses the unique selection of the parse tree for a sentence
  - We should eliminate the ambiguity in the grammar during the design phase of the compiler.
  - We have to prefer one of the parse trees of a sentence (generated by an ambiguous grammar) to disambiguate that grammar to restrict to this choice.

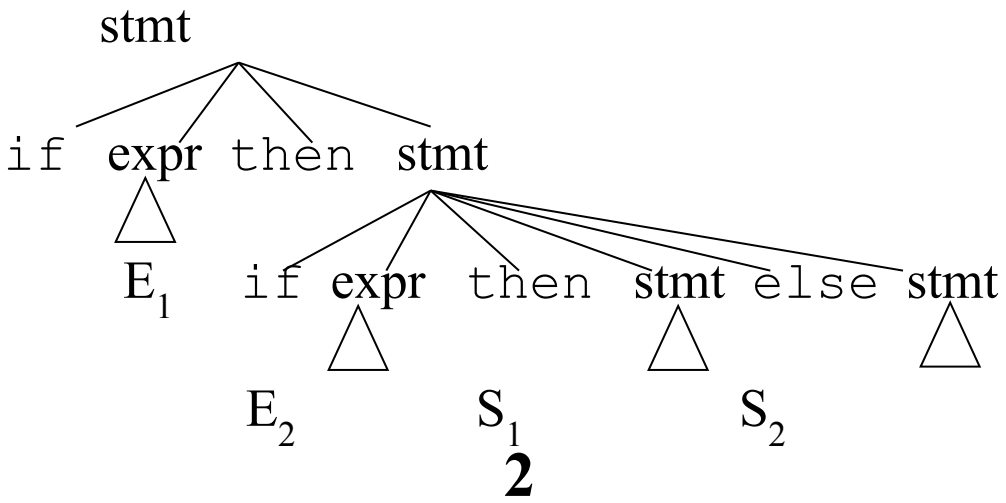
Ambiguity (cont.)

stmt → `if` expr `then` stmt |
`if` expr `then` stmt `else` stmt | otherstmts

`if` E_1 `then` `if` E_2 `then` S_1 `else` S_2



1



2

Ambiguity (cont.)

- We prefer the second parse tree (else matches with closest if).
- So, we have to disambiguate our grammar to reflect this choice.

- The unambiguous grammar will be:

stmt → matchedstmt | unmatchedstmt

matchedstmt → `if` expr `then` matchedstmt `else` matchedstmt
| otherstmts

unmatchedstmt → `if` expr `then` stmt |
`if` expr `then` matchedstmt `else` unmatchedstmt

Ambiguity – Operator Precedence

- Ambiguous grammars (because of ambiguous operators) can be disambiguated according to the precedence and associativity rules.

$E \rightarrow E+E \mid E*E \mid E\hat{}E \mid id \mid (E)$

disambiguate the grammar

precedence: ^ (right to left)

⇩ * (left to right)

\+ (left to right)

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow G\hat{}F \mid G$

$G \rightarrow id \mid (E)$

Operator precedence

Precedence	Operator	Description	Associativity
1	()	Parentheses (function call)	Left-to-Right
	[]	Array Subscript (Square Brackets)	
	->	Structure Pointer Operator	
	++ , —	Postfix increment, decrement	
2	++ / —	Prefix increment, decrement	Right-to-Left
	+ / –	Unary plus, minus	
	! , ~	Logical NOT, Bitwise complement	
3	*, /, %	Multiplication, division, modulus	Left-to-Right
4	+/-	Addition, subtraction	
5	<< , >>	Bitwise shift left, Bitwise shift right	

precedence

Precedence	Operator	Description	Associativity
6	< , <= > , >=	Relational	Left-to-Right
7	== , !=		
8	&&	Logical	
9	||		
14	=, += , -= *= , /= %= , &= ^= , |= <<=, >>=	Assignment operators	Right-to-Left

Left Recursion

- A grammar is ***left recursive*** if it has a non-terminal A such that there is a derivation

  $A \overset{+}{\Rightarrow} A\alpha$ for some string α

- Top-down parsing techniques **cannot** handle left-recursive grammars.
- So, we have to convert left-recursive grammar into an equivalent grammar which is not left-recursive.
- The left-recursion may appear in a single step of the derivation (*immediate left-recursion*), or may appear in more than one step of the derivation.

Eliminate Left-Recursion

$A \rightarrow A\,\alpha \mid \beta$ where β does not start with A

$\Downarrow$ eliminate immediate left recursion

$A \rightarrow \beta\,A'$

$A' \rightarrow \alpha\,A' \mid \varepsilon$ an equivalent grammar

Immediate Left-Recursion - Example

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow id \mid (E)$

$\Downarrow$ eliminate immediate left recursion

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow id \mid (E)$

Immediate Left-Recursion

In general,

$A \rightarrow A\,\alpha_1 \mid \ldots \mid A\,\alpha_m \mid \beta_1 \mid \ldots \mid \beta_n$ where $\beta_1 \ldots \beta_n$ do not start with A

$\Downarrow$ eliminate immediate left recursion

$A \rightarrow \beta_1\,A' \mid \ldots \mid \beta_n\,A'$

$A' \rightarrow \alpha_1\,A' \mid \ldots \mid \alpha_m\,A' \mid \varepsilon$ an equivalent grammar

Left Recursion Elimination Practice

1. A → ABd / Aa / a
   B → Be / b

2. E → E + T / T
   T → T x F / F
   F → id

3. S → A
   A → Ad / Ae / aB / ac
   B → bBc / f

These are all immediate left recursion problems!!

Left-Recursion -- Problem

- A grammar cannot be immediately left-recursive, but it still can be left-recursive.
- By just eliminating the immediate left-recursion, we may not get a grammar which is not left-recursive.

$S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$ This grammar is not immediately left-recursive, but it is still left-recursive. Indirect Left Recursion

$S \Rightarrow Aa \Rightarrow Sda$ or

$A \Rightarrow Sd \Rightarrow Aad$ causes to a left-recursion

So, we have to eliminate all left-recursions from this grammar

Eliminate Left-Recursion -- Algorithm

- Arrange non-terminals in some order: $A_1 \ldots A_n$

  - **for** i **from** 1 **to** n **do** {

    - **for** j **from** 1 **to** i-1 **do** {

      replace each production

      $A_i \rightarrow A_j \gamma$ by

      $A_i \rightarrow \alpha_1 \gamma \mid \ldots \mid \alpha_k \gamma$

      where $A_j \rightarrow \alpha_1 \mid \ldots \mid \alpha_k$

    }

    - eliminate immediate left-recursions among A_i productions

  }

- Will not work for a grammar which has cycle, $A \Rightarrow A$.
- And for ε productions.

Eliminate Left-Recursion -- Example

S → Aa | b
A → Ac | Sd | f

- Order of non-terminals: S, A

for S:
- we do not enter the inner loop.
- there is no immediate left recursion in S.

for A:

- Replace A → Sd with A → Aad | bd
  So, we will have A → Ac | Aad | bd | f
- Eliminate the immediate left-recursion in A

A → bdA′ | fA′
A’ → cA′ | adA′ | ε

So, the resulting equivalent grammar which is not left-recursive is:

S → Aa | b
A → bdA′ | fA′
A’ → cA′ | adA′ | ε

Eliminate Left-Recursion – Example2

A → Ac | Sd | f
S → Aa | b

- Order of non-terminals: A, S

for A:
- we do not enter the inner loop.
- Eliminate the immediate left-recursion in A
  A → SdA' | fA'
  A' → cA' | ε

for S:
- Replace S → Aa with S → SdA'a | fA'a
  So, we will have S → SdA'a | fA'a | b
- Eliminate the immediate left-recursion in S
  S → fA'aS' | bS'
  S' → dA'aS' | ε

So, the resulting equivalent grammar which is not left-recursive is:

S → fA'aS' | bS'
S' → dA'aS' | ε
A → SdA' | fA'
A' → cA' | ε

Left-Factoring

- A predictive parser (a top-down parser without backtracking) insists that the grammar must be *left-factored*.

  stmt → `if` expr `then` stmt `else` stmt |
  `if` expr `then` stmt

- when we see `if`, we cannot know which production rule to choose to re-write ***stmt*** in the derivation.

Left-Factoring (cont.)

- In general,

  $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$

  where α is non-empty and the first symbols of β_1 and β_2 (if they have one) are different.

- when processing α we cannot know whether expand

  A to $\alpha\beta_1$ or A to $\alpha\beta_2$

- But, if we re-write the grammar as follows

  $A \rightarrow \alpha A'$

  $A' \rightarrow \beta_1 \mid \beta_2$

  so, we can immediately expand A to $\alpha A'$

Left-Factoring -- Algorithm

For each non-terminal A with two or more alternatives (production rules) with a common non-empty prefix,

$$A \rightarrow \alpha\beta_1 \mid \ldots \mid \alpha\beta_n \mid \gamma_1 \mid \ldots \mid \gamma_m$$

convert it into

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \ldots \mid \gamma_m$$
$$A' \rightarrow \beta_1 \mid \ldots \mid \beta_n$$

Left-Factoring – Example 1

$A \rightarrow \underline{a}bB \mid \underline{a}B \mid cdg \mid cdeB \mid cdfB$

$\Downarrow$

$A \rightarrow aA' \mid \underline{cdg} \mid \underline{cde}B \mid \underline{cdf}B$

$A' \rightarrow bB \mid B$

$\Downarrow$

$A \rightarrow aA' \mid cdA''$

$A' \rightarrow bB \mid B$

$A'' \rightarrow g \mid eB \mid fB$

Left-Factoring – Example 2

$A \rightarrow ad \mid a \mid ab \mid abc \mid b$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid b \mid bc$

$\Downarrow$

$A \rightarrow aA' \mid b$

$A' \rightarrow d \mid \varepsilon \mid bA''$

$A'' \rightarrow \varepsilon \mid c$


End of Part 1